Practical: 6

Date: 6/10/2026

Bias Variance Trade-off

Procedure

1. Import the required Python libraries such as NumPy, Matplotlib, Scikit-learn and Seaborn.
2. Define a function to create polynomial regression models using PolynomialFeatures and LinearRegression.
3. Generate a sample dataset using the make_data() function.
4. Fit polynomial regression models of degrees 1, 3 and 5 to the generated data.
5. Plot the observed data points along with the fitted polynomial regression curves.
6. Generate polynomial degrees from 0 to 20 and use the validation_curve() function to calculate training and validation scores.
7. Plot the training and validation scores against the polynomial degree for 50 observations.
8. Generate a second dataset containing 200 observations.
9. Calculate the training and validation scores for the second dataset.
10. Plot the results for 200 observations and compare them with the results obtained from 50 observations.
11. Display all the graphs separately using Matplotlib.

Solution:

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
def PolynomialRegression(degree=2, **kwargs):
    return make_pipeline(PolynomialFeatures(degree),LinearRegression (**kwargs))

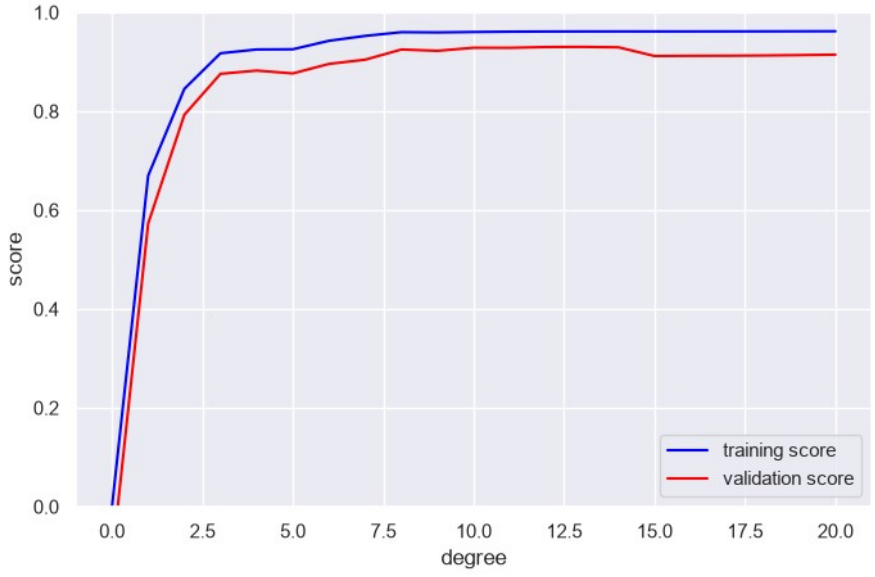
import numpy as np
def make_data(N, err=1.0, rseed=1):
    rng = np.random.RandomState(rseed)
    X = rng.rand(N, 1) ** 2
    y = 10 - 1. / (X.ravel() + 0.1)
    if err > 0:
        y += err * rng.randn(N)
    return X, y
X, y = make_data(50)

import matplotlib.pyplot as plt
```

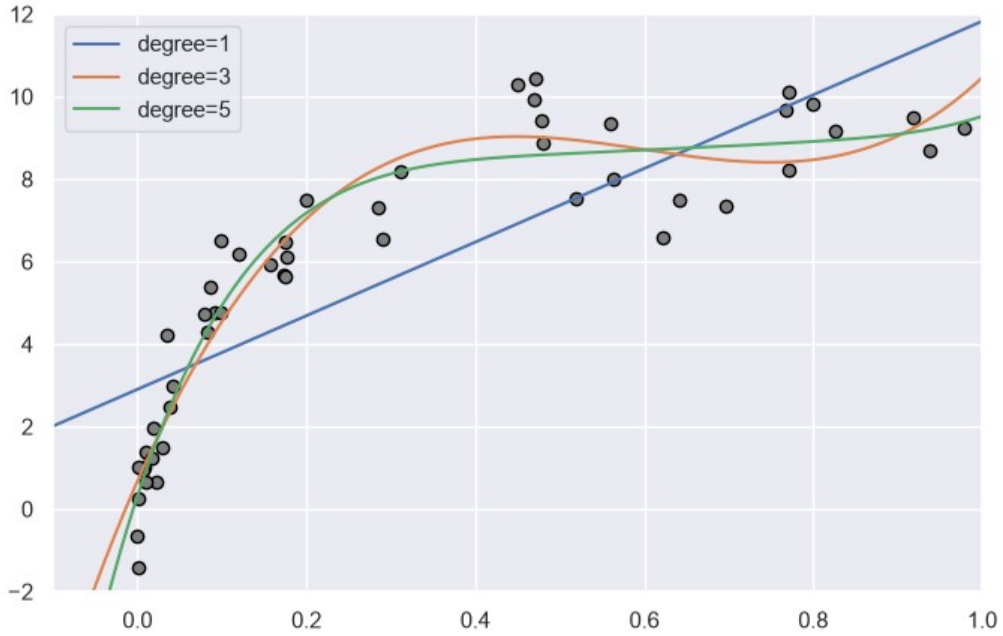
```
import seaborn
seaborn.set()

X_test = np.linspace(-0.1, 1.1, 500)[:, None]
plt.figure(figsize=(8, 5))
plt.scatter(X.ravel(), y, edgecolor='black', color='gray')
axis = plt.axis()
for degree in [1, 3, 5]:
    y_test = PolynomialRegression(degree).fit(X, y).predict(X_test)
    plt.plot(X_test.ravel(), y_test,
             label='degree={0}'.format(degree))
plt.xlim(-0.1, 1.0)
plt.ylim(-2, 12)
plt.legend(loc='best')

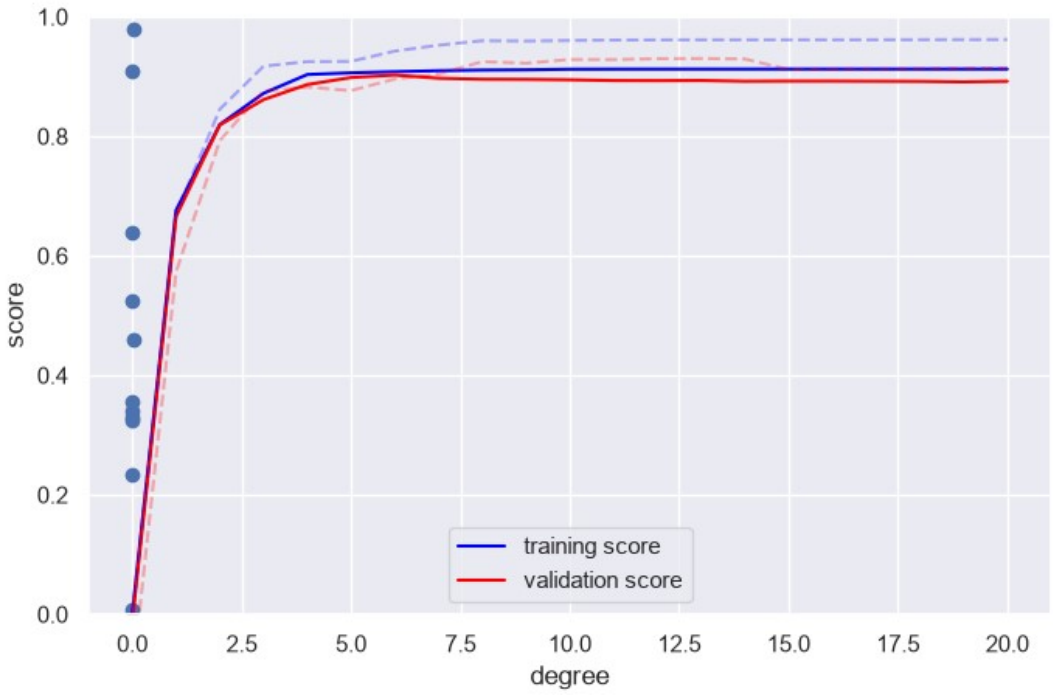
from sklearn.model_selection import validation_curve
degree = np.arange(0, 21)
train_score, val_score = validation_curve
(PolynomialRegression(), X,  y,param_name='polynomialfeatures__degree',
    param_range=degree,  cv=7)
plt.figure(figsize=(8, 5))
plt.plot(degree, np.median(train_score, 1),color='blue', label='training score')
plt.plot(degree, np.median(val_score, 1),color='red', label='validation score')
plt.legend(loc='best')
plt.ylim(0, 1)
plt.xlabel('degree')
plt.ylabel('score')
```



```
X2, y2 = make_data(200)
plt.figure(figsize=(8, 5))
plt.scatter(X2.ravel(), y2)
```




```
degree = np.arange(21)
train_score2, val_score2 = validation_curve(PolynomialRegression(), X2, y2,
    param_name='polynomialfeatures__degree',param_range=degree,cv=7)
plt.plot(degree, np.median(train_score2, 1),color='blue', label='training score')
plt.plot(degree, np.median(val_score2, 1),color='red', label='validation score')
plt.plot(degree, np.median(train_score, 1),color='blue', alpha=0.3,
linestyle='dashed')
plt.plot(degree, np.median(val_score, 1),color='red', alpha=0.3,
linestyle='dashed')
plt.legend(loc='lower center')
plt.ylim(0, 1)
plt.xlabel('degree')
plt.ylabel('score')
plt.show()
```


1.0
0.8
0.6
0.4
0.2
0.0
score
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
degree
training score
validation score